ESP32-S3-Nano

From Waveshare Wiki
Jump to: navigation, search

Overview

Introduction

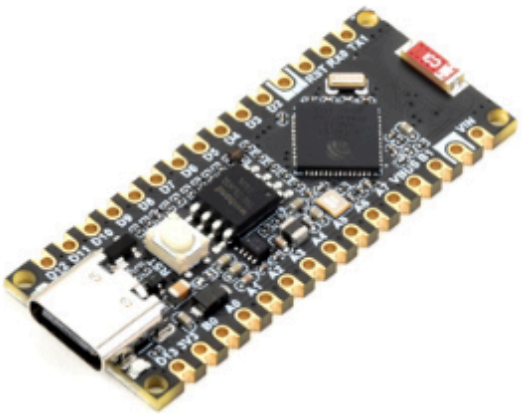
ESP32-S3-Nano adopts ESP32-S3R8 as the main controller, compatible with Arduino Nano ESP32, and is suitable for applications such as IoT or MicroPython. Compact in size while with powerful performance, it is applicable for the standalone project.

Features

- Adopting ESP32-S3R8 as the main controller with Xtensa® 32-bit LX7 dual-core processor, capable of running at 240 MHz.
- Integrated 512KB SRAM, 384KB ROM, 8MB PSRAM, 16MB Flash memory.
- Integrated 2.4GHz Wi-Fi and Bluetooth LE dual-mode wireless communication, with superior RF performance.
- Supports seamless switching between Arduino and MicroPython programming, offering greater flexibility.
- Compatible with Arduino IoT Cloud, enabling users to monitor and control their projects from anywhere with Arduino Internet of Things (IoT) cloud applications.

- Supports HID (Human Interface Device), emulating Human Interface Devices such as keyboards or mice via USB port for easier interaction with PC.

Version Options

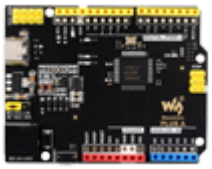
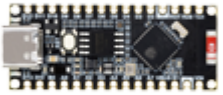


ESP32-S3-Nano

ESP32-S3-Nano-M
with pre-soldered header (black)

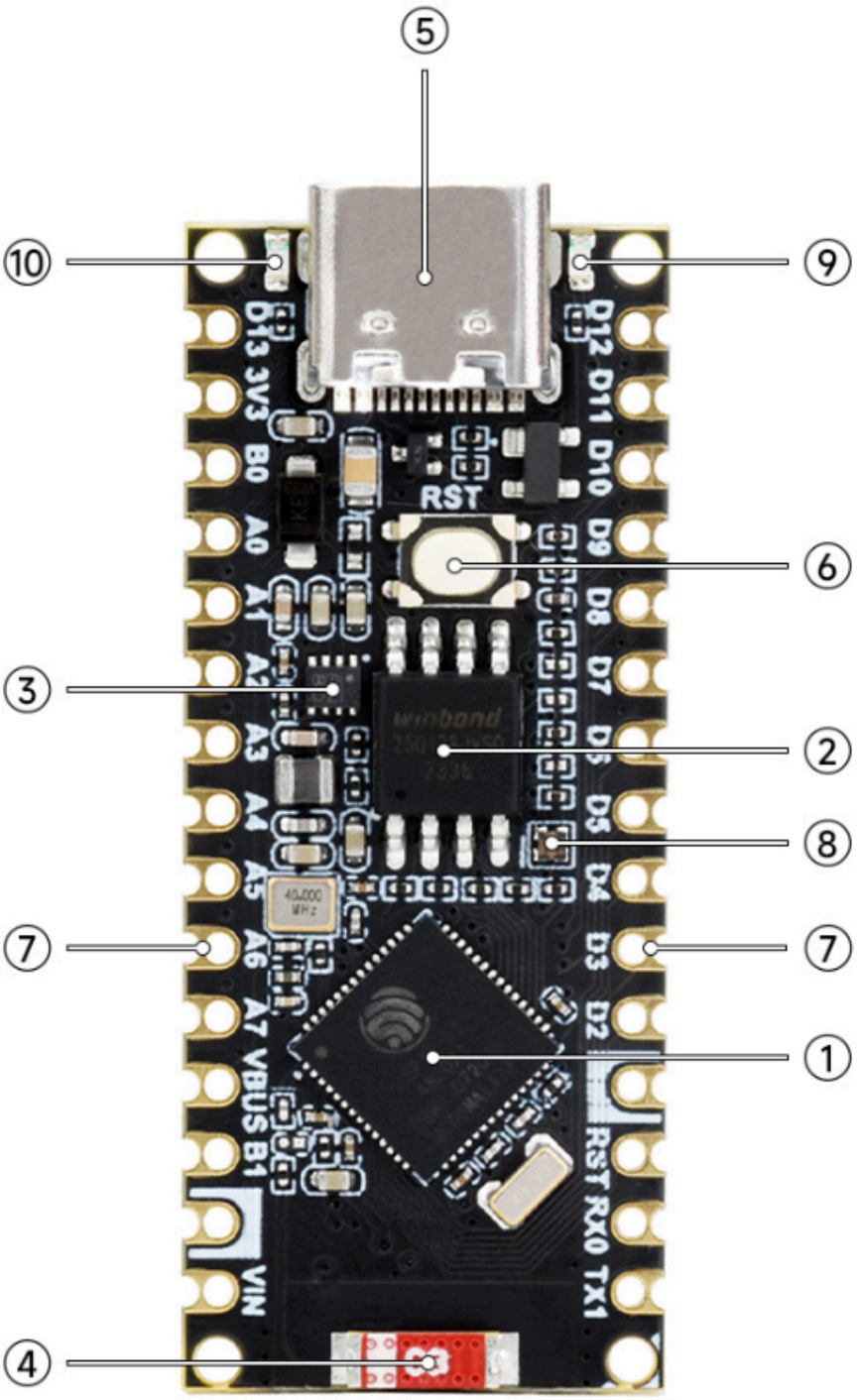
(/wiki/File:ESP32-S3-Nano_Version.png)

Product Parameters Comparison

MODEL	 (/wiki/File:R7FA4-PLUS-A-10.png)	 (/wiki/File:R7FA4-PLUS-B-10.png)	 (/wiki/File:ESP32-S3-Nano-10.png)
MICROCONTROLLER	R7FA4 (32-bit ARM Cortex-M4)	R7FA4 (32-bit ARM Cortex-M4) ESP32-S3FN8 (Dual-core 32-bit Xtensa LX7)	ESP32-S3R8 (Dual-core 32-bit Xtensa LX7)
CLOCK FREQUENCY	R7FA4: 48MHz	R7FA4: 48MHz ESP32-S3FN8: 240MHz	ESP32-S3R8: 240MHz
STORAGE	R7FA4: 256kB Flash, 32kB RAM	R7FA4: 256kB FLASH, 32kB RAM ESP32-S3FN8: 384kB ROM, 512kB RAM, 8MB Flash	ESP32-S3R8: 384kB ROM, 512kB RAM, 16MB Flash, 8MB PSRAM
WIRELESS COMMUNICATION	None	2.4GHz WiFi + Bluetooth LE	
OPERATING VOLTAGE	Options for 5V/3.3V, support more shields		3.3V
POWER INPUT	6~24V		6~21V
RESET BUTTON	Lateral, easier to use when connecting with shield		Vertical
IO PIN OUTPUT CURRENT	8mA		40mA
DIGITAL PINS	14		14
ANALOG PINS	6		8
DAC	2		None
PWM	6		5
UART	1		2
I2C	1		1
SPI	1		1
CAN	1		None
DC JACK	Low profile, shields won't be blocked anymore while connecting		None
POWER OUTPUT HEADER	Provides 5V OR 3.3V power output and common-grounding with other boards		None

5V POWER OUTPUT	Up to 2000mA Max, features higher driving capability	1000mA Max
EXPERIMENTAL BOARD	Support, the solder pad is provided for DIY interfaces to connect with the experimental board	Support

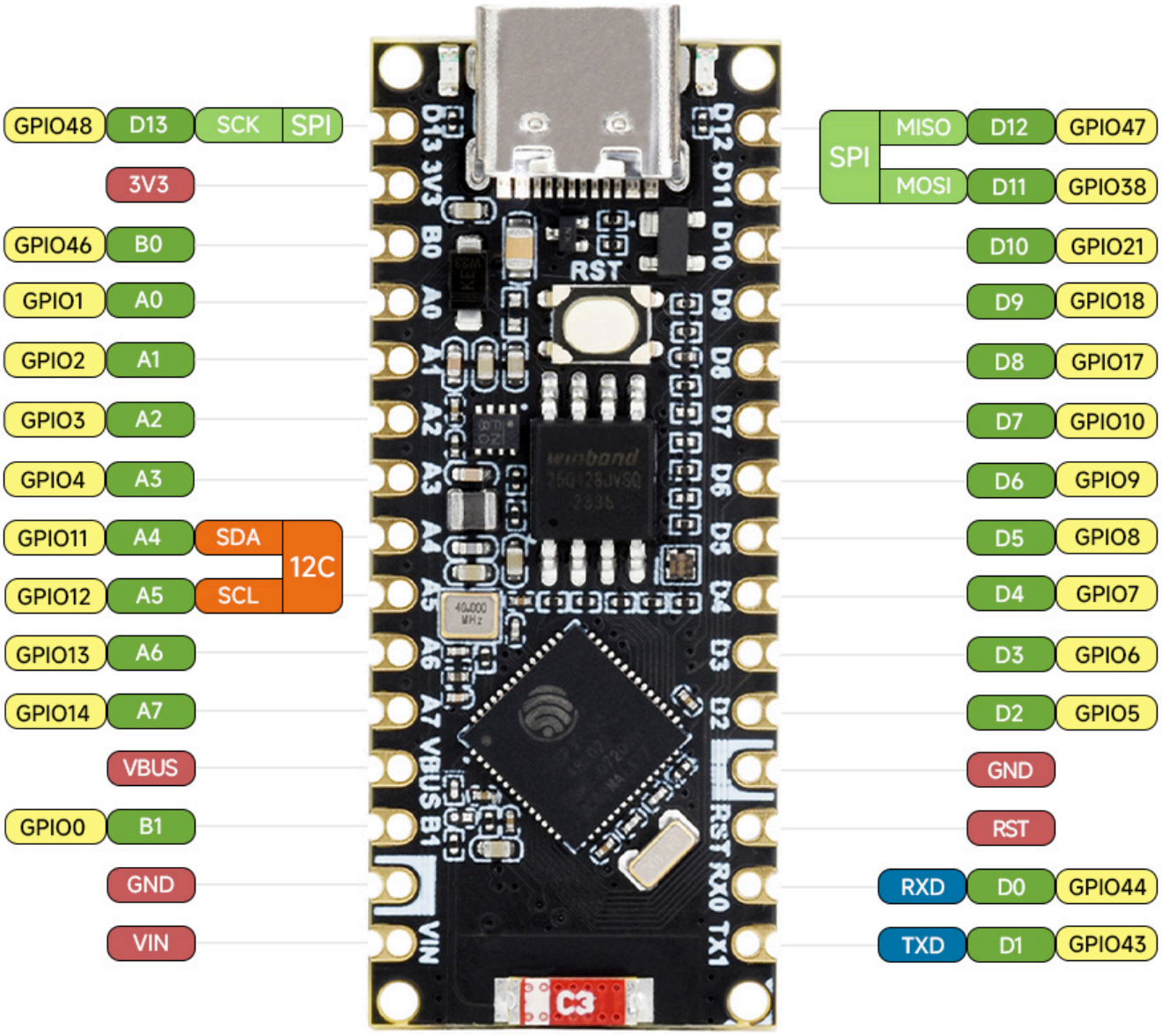
Hardware Description



1. **ESP32-S3R8 dual-core processor**
   up to 240 MHz running frequency
2. **W25Q128JVSIQ**
   16MB Flash, for program and data storage
3. **MP2322GQH**
   3.3V voltage regulator
4. **2.4G ceramic antenna**
5. **USB Type-C connector**
   for downloading program and serial port debugging
6. **RST button**
   for resetting ESP32-S3R8
7. **Arduino Nano interface**
   compatible with Arduino interface, adapting 2.54 pitch solder pad, can be directly connected to experimental board
8. **RGB indicator**
   blinks and then turns off during power on or reset, supports programmable control after the board has booted up normally
9. **Power indicator**
10. **User LED**

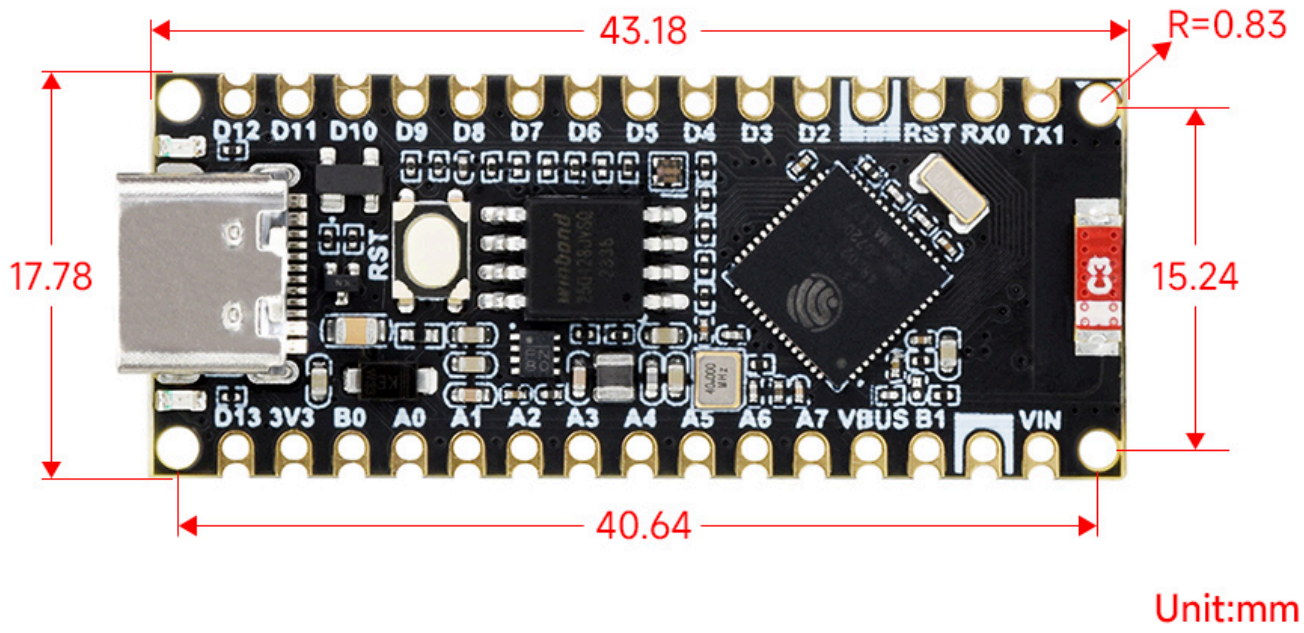
(/wiki/File:ESP32-S3-Nano_Version2.png)

Pinout Definition



(/wiki/File:ESP32-S3-Nano_Version3.jpg)

Dimensions



(/wiki/File:ESP32-S3-Nano_Version4.jpg)

User Guide

Environment Setting

The software framework for ESP32 series development boards is mature, and you can use CircuitPython, MicroPython and C/C++ (Arduino, ESP-IDF) for rapid prototyping of product development. Here's a brief introduction to these three development approaches:

- CircuitPython is a programming language designed to simplify coding tests and learning on low-cost microcontroller boards. It is an open-source derivative of the MicroPython programming language, primarily aimed at students and beginners. CircuitPython development and maintenance are supported by Adafruit Industries.
  - You can refer to development documentation (https://docs.circuitpython.org/en/latest/shared-bindings/index.html) for CircuitPython-related applications development.
  - The GitHub (https://github.com/adafruit/Adafruit_CircuitPython_Bundle) library for CircuitPython allows for recompilation for custom development.
- MicroPython is an efficient implementation of the Python 3 programming language. It includes a small subset of the Python standard library and has been optimized to run on microcontrollers and resource-constrained environments.
  - You can refer to development documentation (https://docs.micropython.org/en/latest/) for MicroPython-related application development.

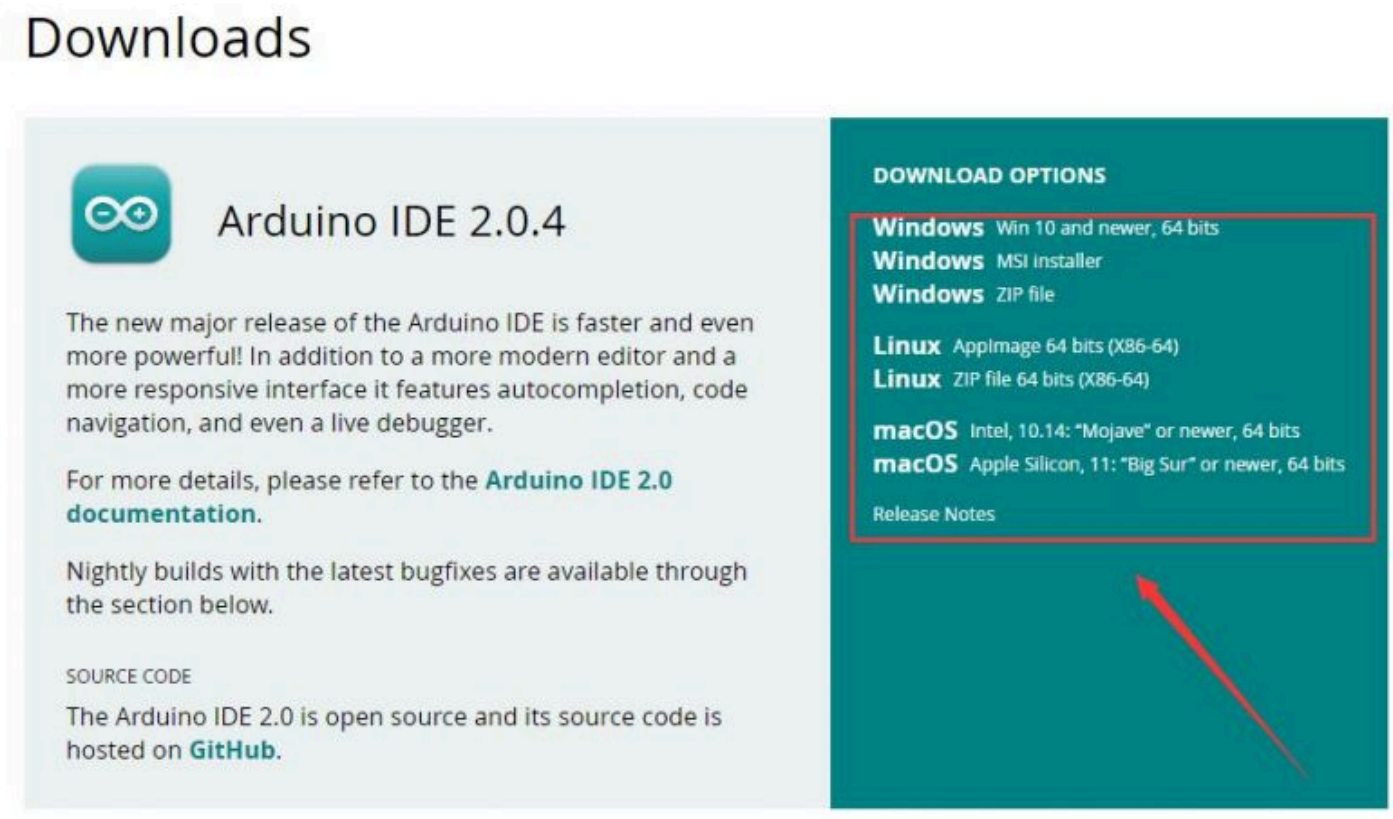
    - The GitHub library (https://github.com/micropython/micropython) for MicroPython allows for recompilation for custom development.
- The official libraries and support from Espressif Systems for C/C++ development make it convenient for rapid installation.
    - Arduino development manual (https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html) for ESP32 series
    - ESP-IDF development manual (https://docs.espressif.com/projects/esp-idf/en/stable/esp32s2/get-started/index.html) for ESP32 series
- The environment is set up under Windows 10, users can choose to use Arduino or Visual Studio Code (ESP-IDF) as IDE for development. For Mac/Linux OS users please refer to the official instructions (https://docs.espressif.com/projects/esp-idf/en/latest/esp32/get-started/index.html).

Arduino

Install Arduino IDE

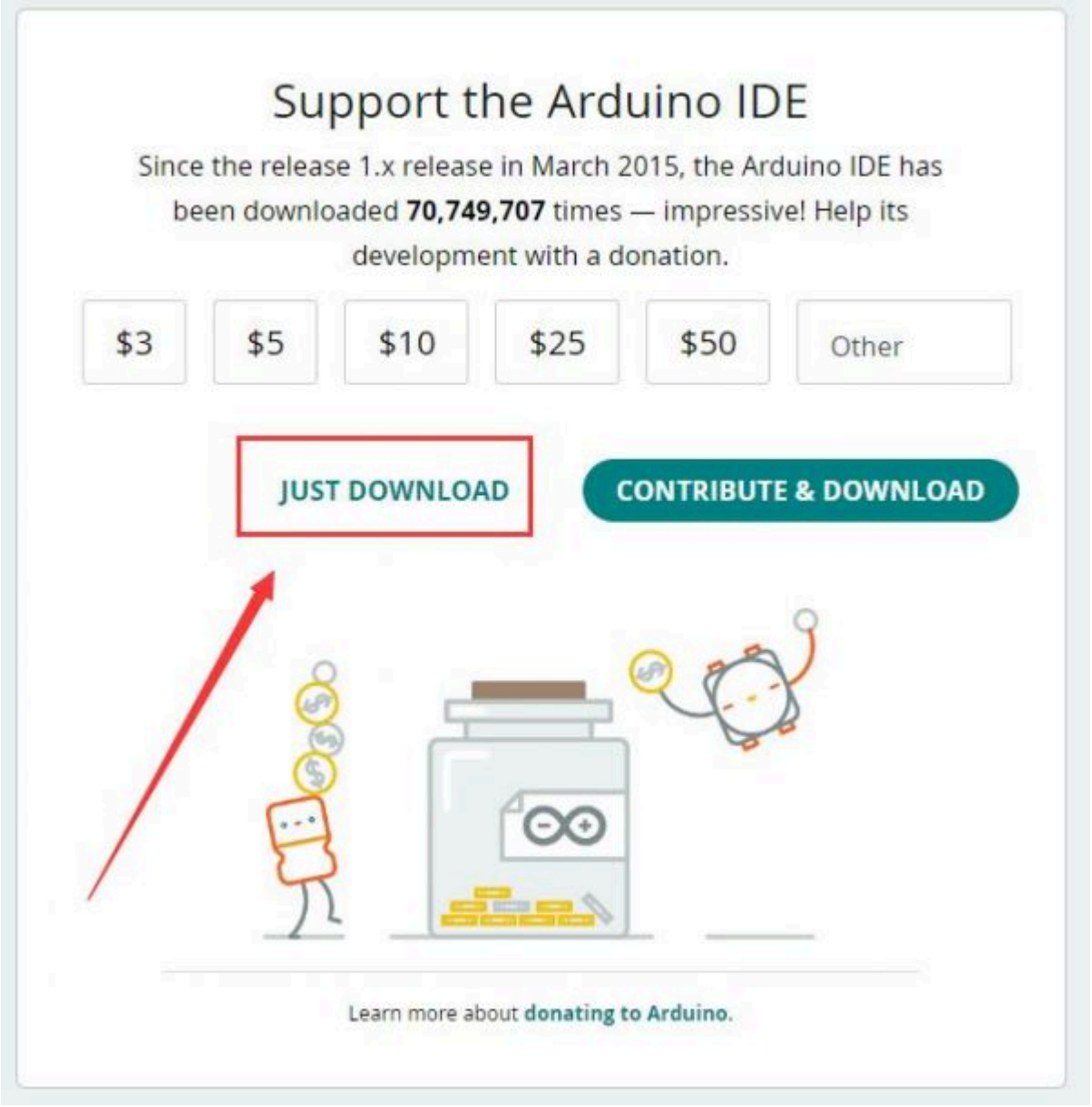
- The following development system is Windows.

1. Open the official software download webpage (https://www.arduino.cc/en/software), and according to the corresponding system and system bits to download.



(/wiki/File:ESP32-S3-Pico_35.jpg)

2. You can choose "Just Download", or "Contribute & Download".

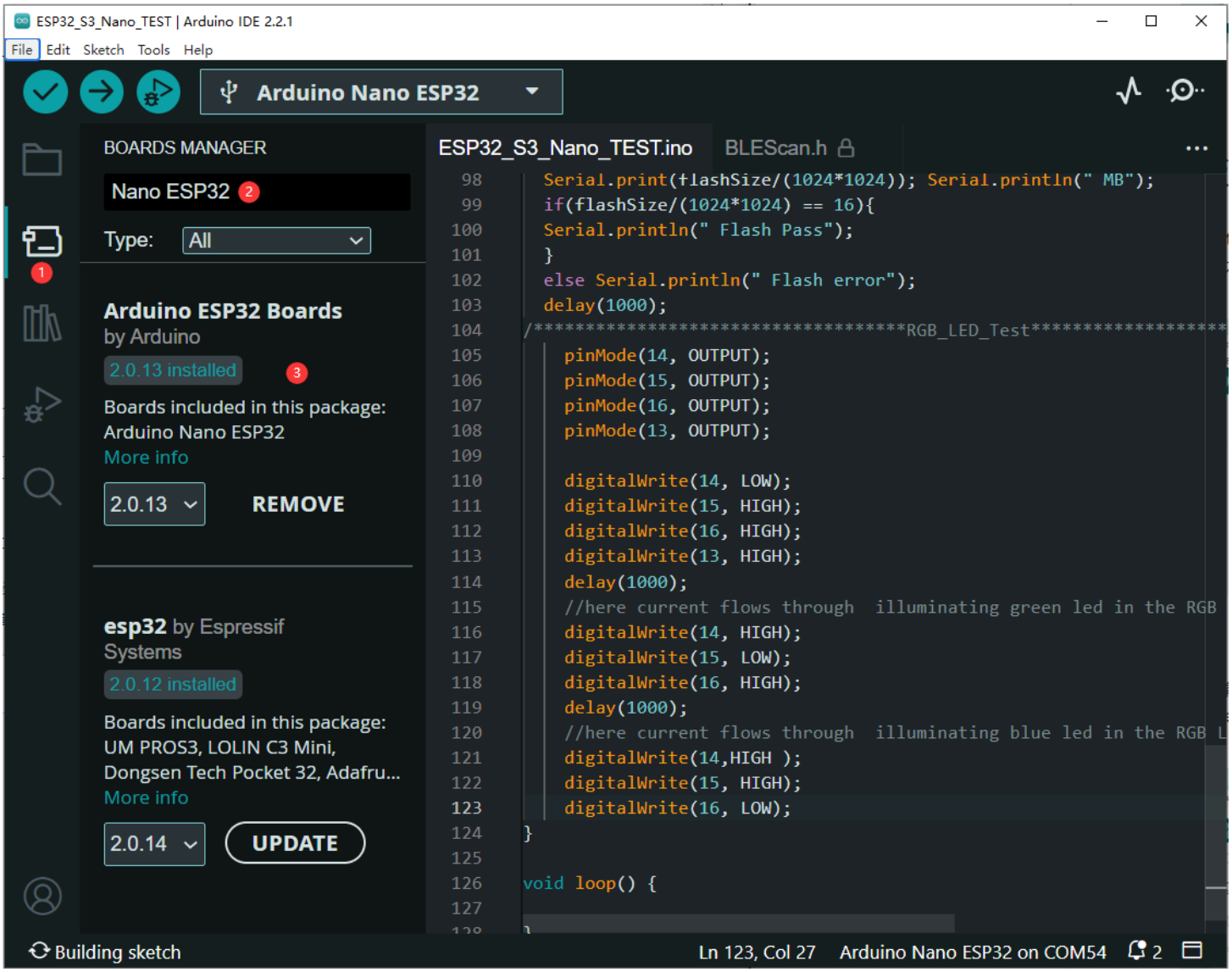


(/wiki/File:ESP32-S3-Pico_36.jpg)

3. Run to install the program and install it all by default.

Install Nano ESP32 Package

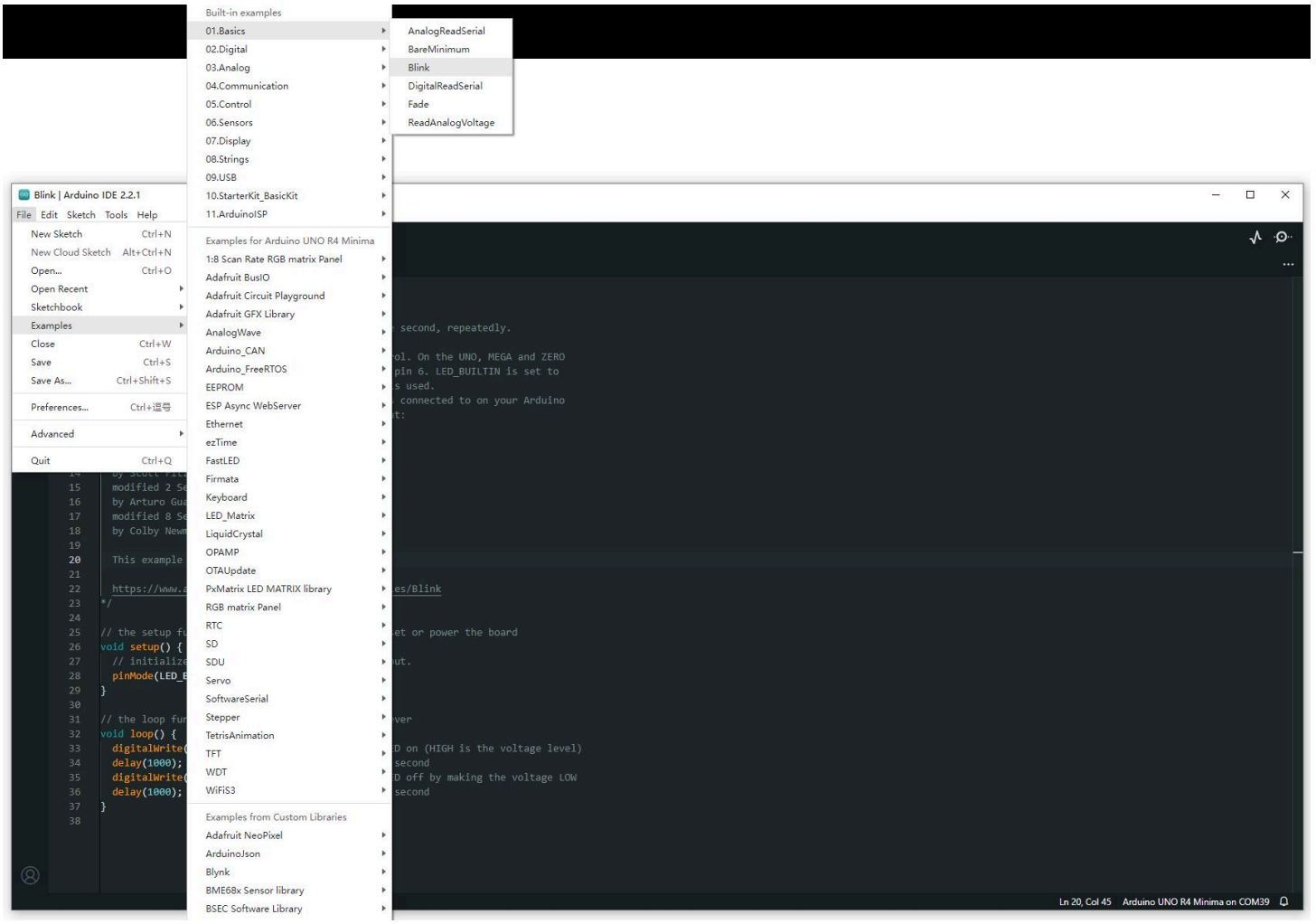
- Install Nano ESP32: Open Boards Manger -> Search "Nano ESP32" and install the latest version (or the version to use).



(/wiki/File:ESP32-S3-Nano_TEST_01.png)

Create Example

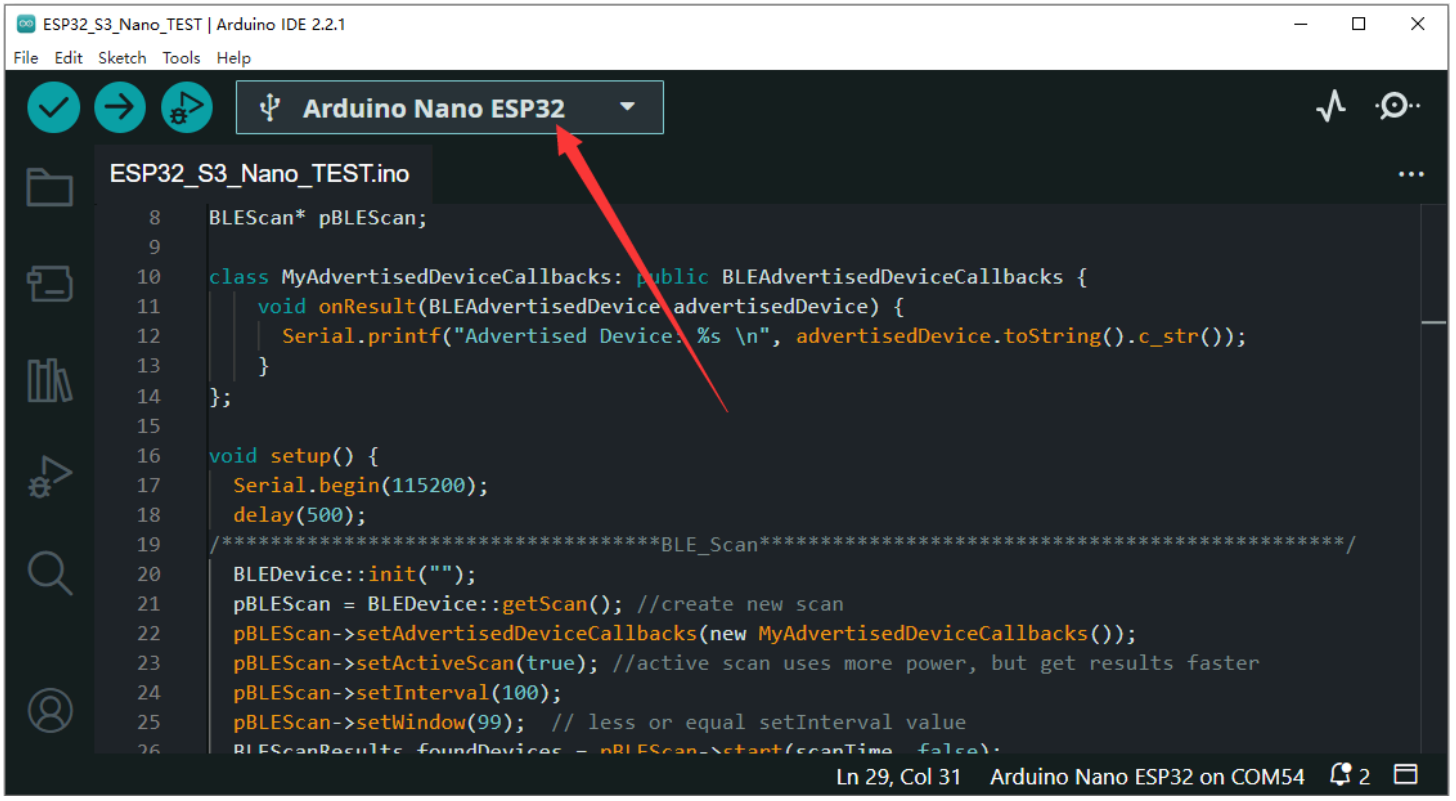
- The following example is about how to make LED blinking. (File -> examples -> Blink under 01.Basics)



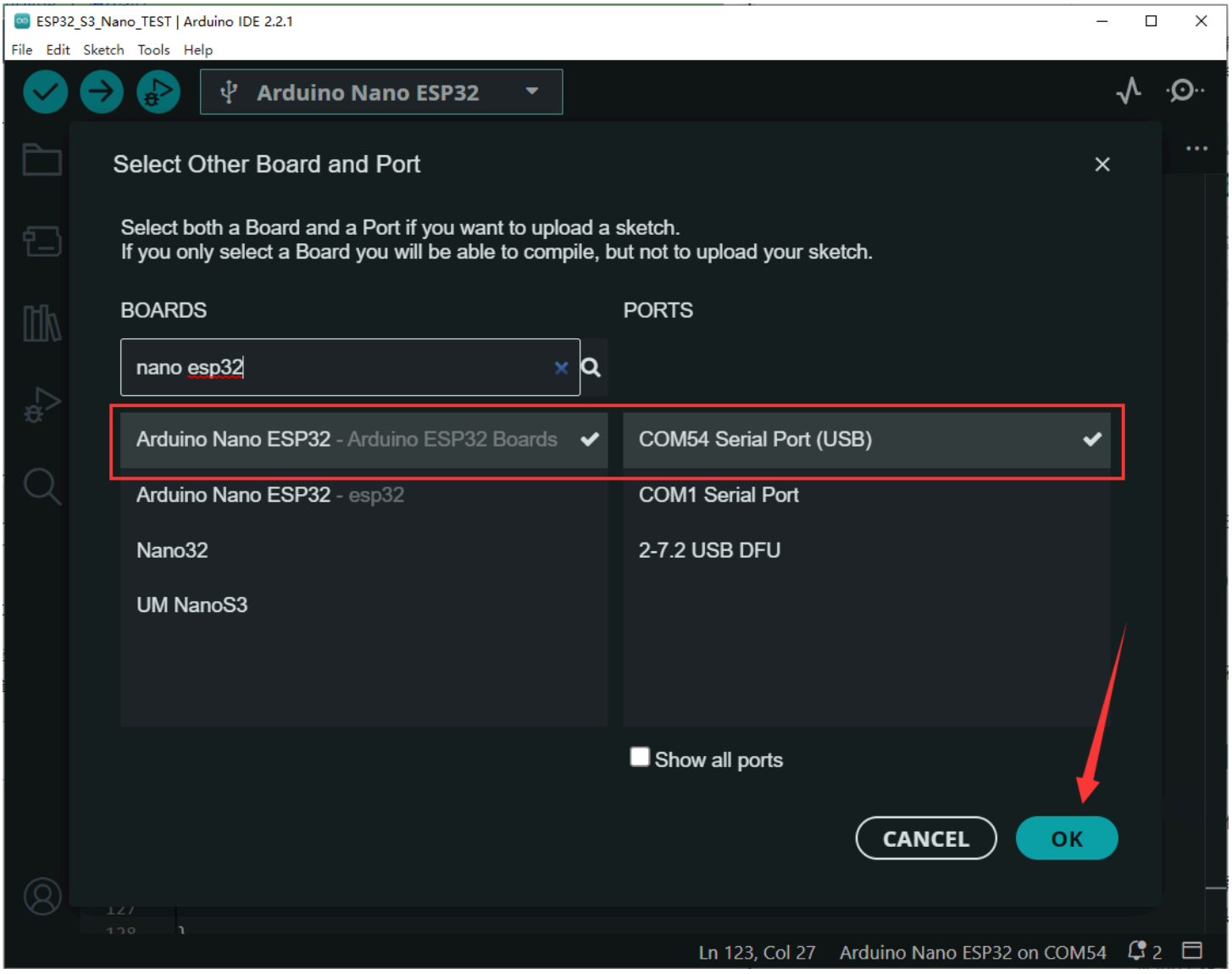
(/wiki/File:R7FA4_PLUS_B_Example.jpg)

- Select the development board and COM ports.

Search "Nano ESP32", select "Arduino Nano ESP32", and then click on OK (the following picture is for reference only, you need to select the corresponding board.)

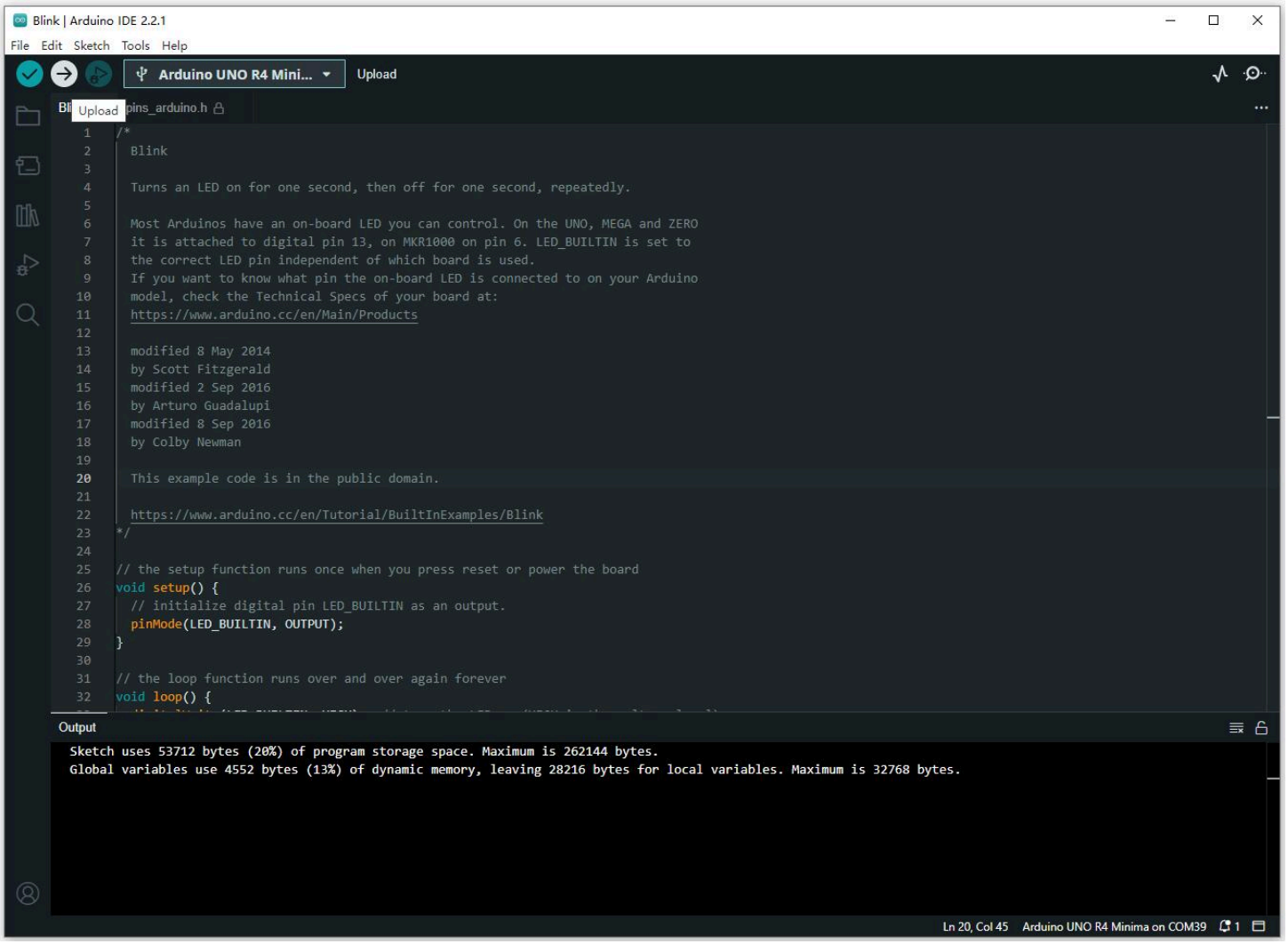


(/wiki/File:ESP32-S3-Nano_TEST_12.png)



(/wiki/File:ESP32-S3-Nano_TEST_02.png)

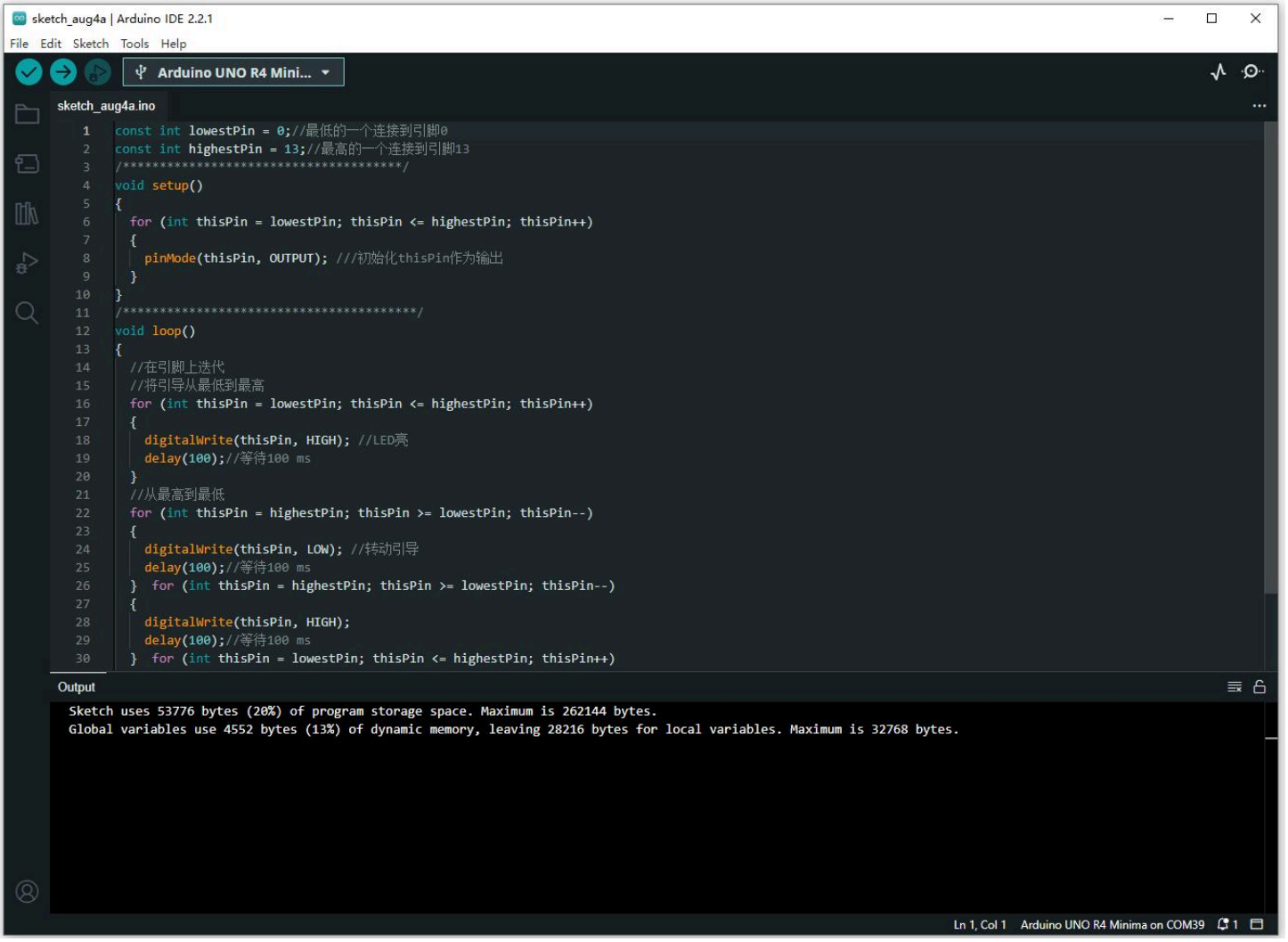
- Click ✓ in the menu bar to compile and → to burn the compiled demo to the board.



(/wiki/File:R7FA4_PLUS_B_Example3.jpg)

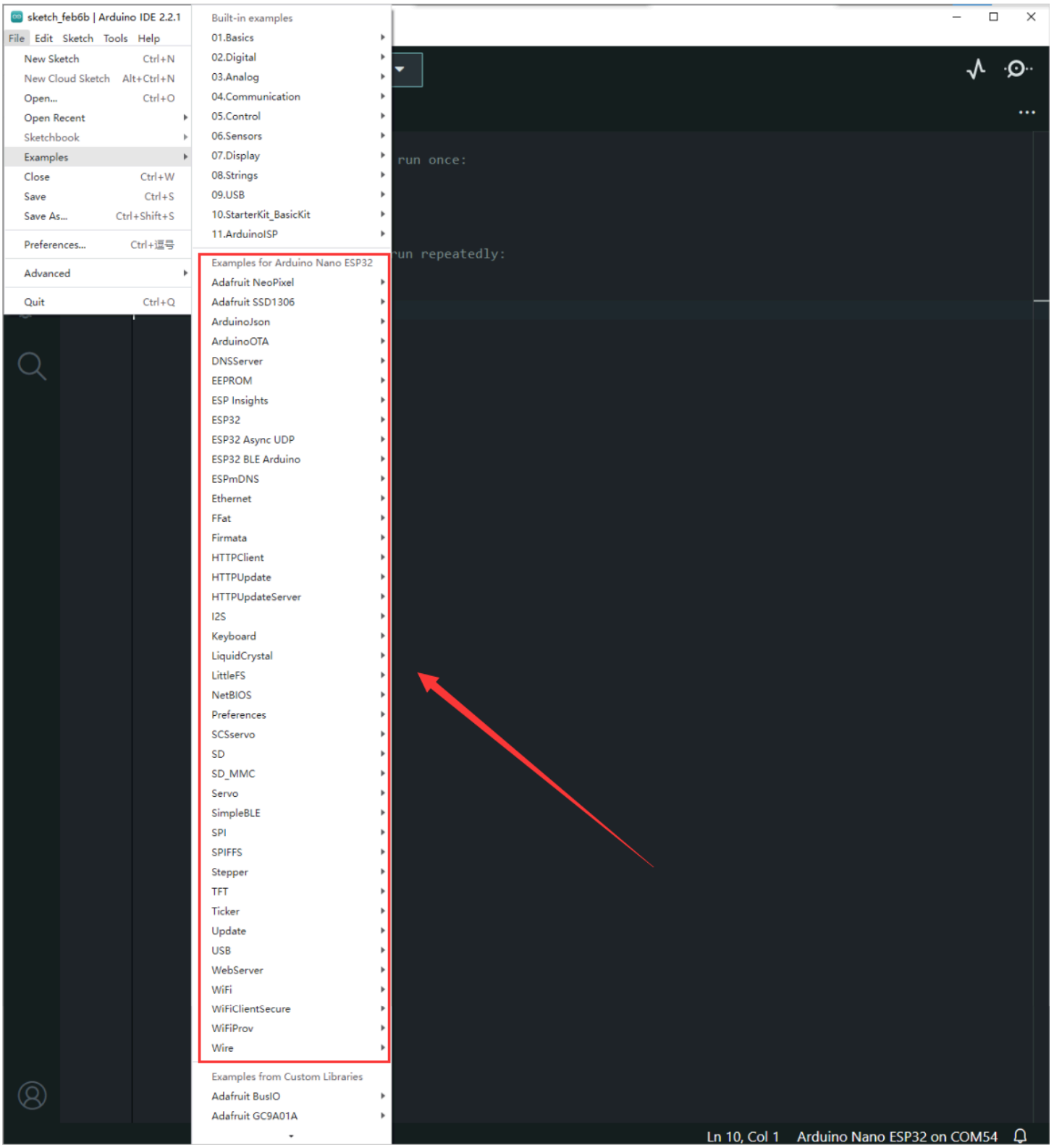
Open Example

- Open the existing example, it is easier to operate. Directly run the ".ino" demo and refer to the operation above, and select the corresponding board and COM port to compile, download and burn.



(/wiki/File:R7FA4_PLUS_A_Open.jpg)

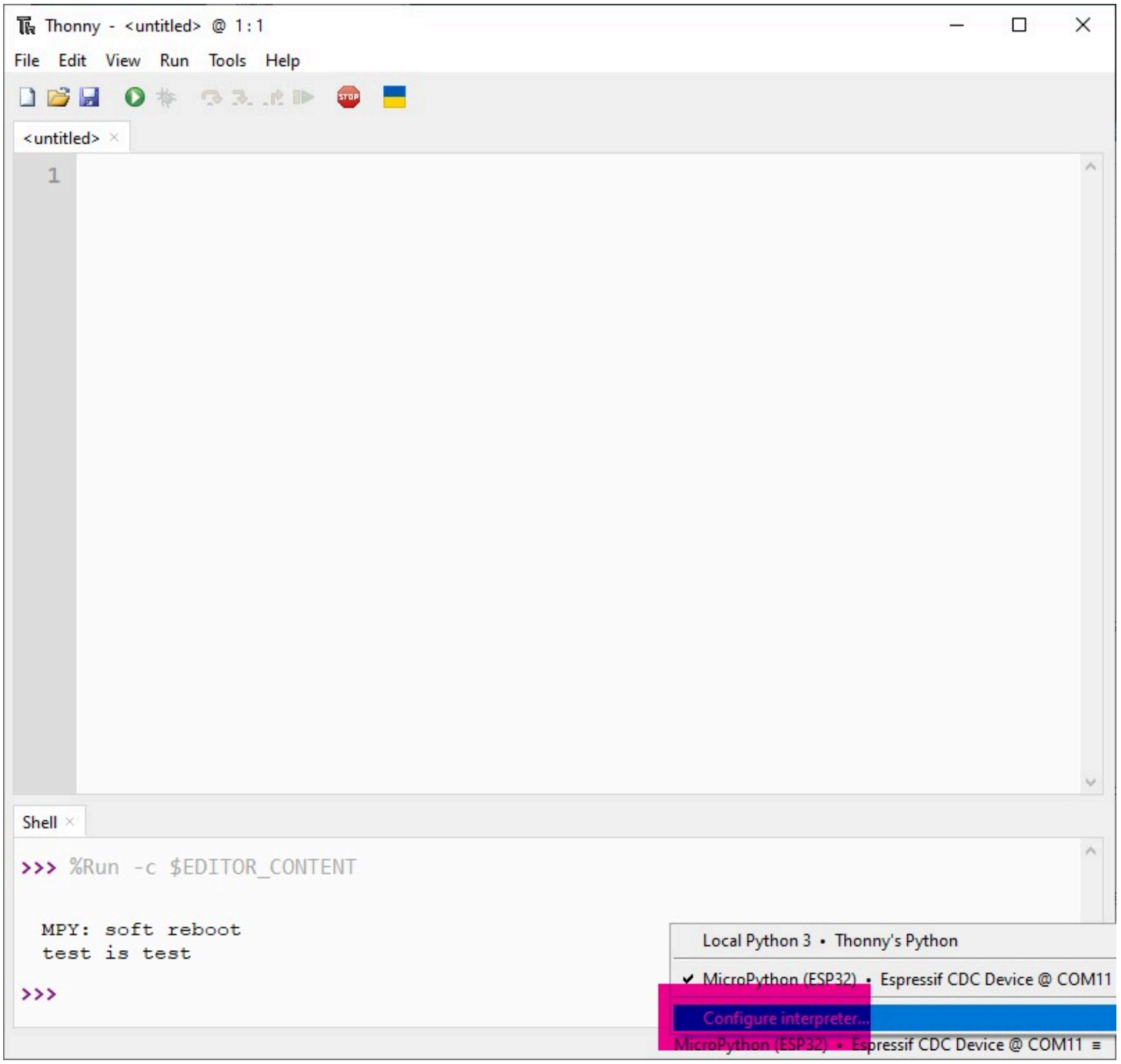
- ESP32-S3-Nano opens the Arduino demo: Open File -> Examples. These demos can be directly used without other external libraries.



(/wiki/File:ESP32-S3-Nano_01.png)

MicroPython

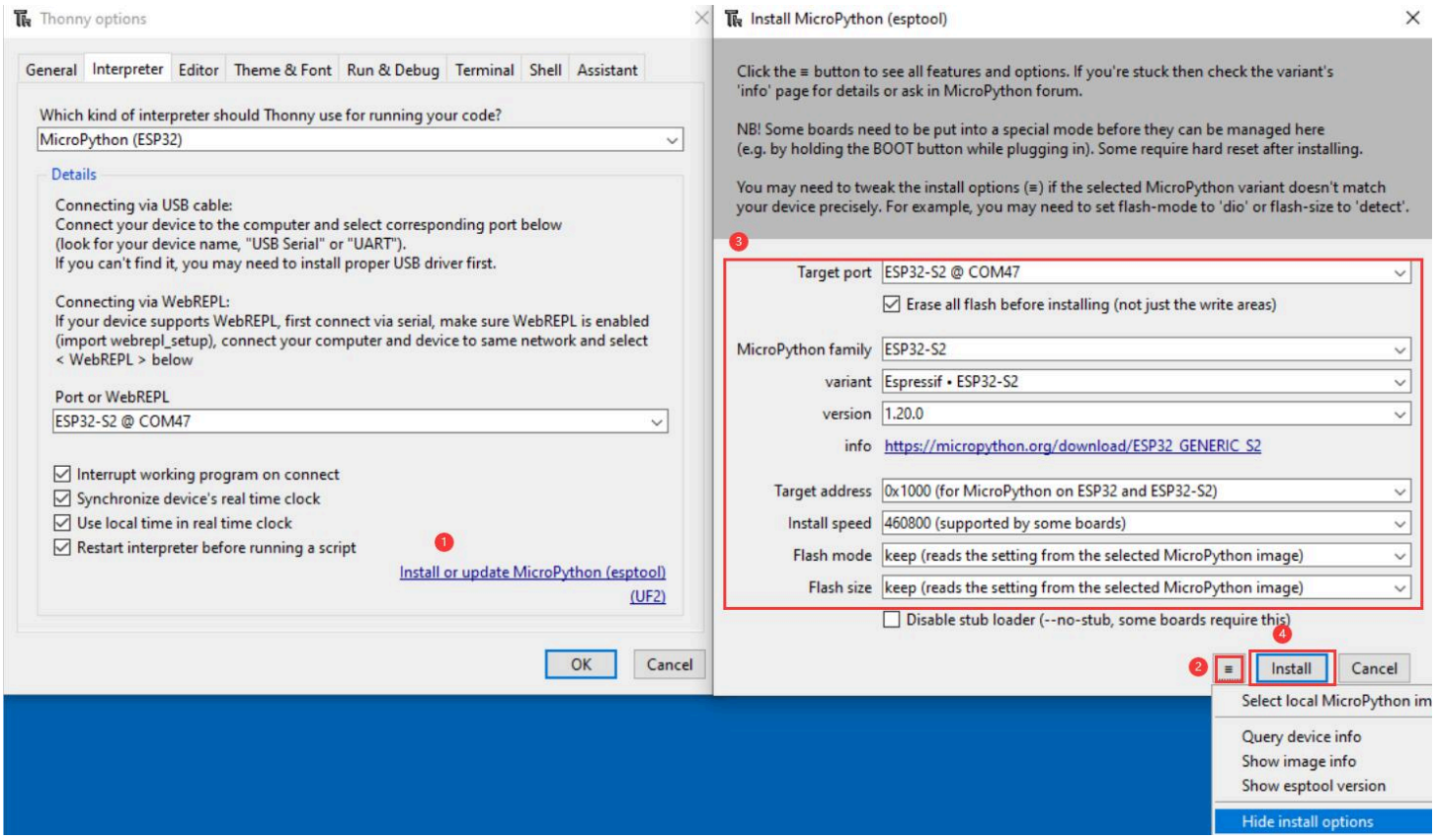
1. Download and install the latest Thonny (https://thonny.org/) IDE, open Thonny IDE -> Configure interpreter... as shown below:



(/wiki/File:CircuitPython_Thonny06.jpg)

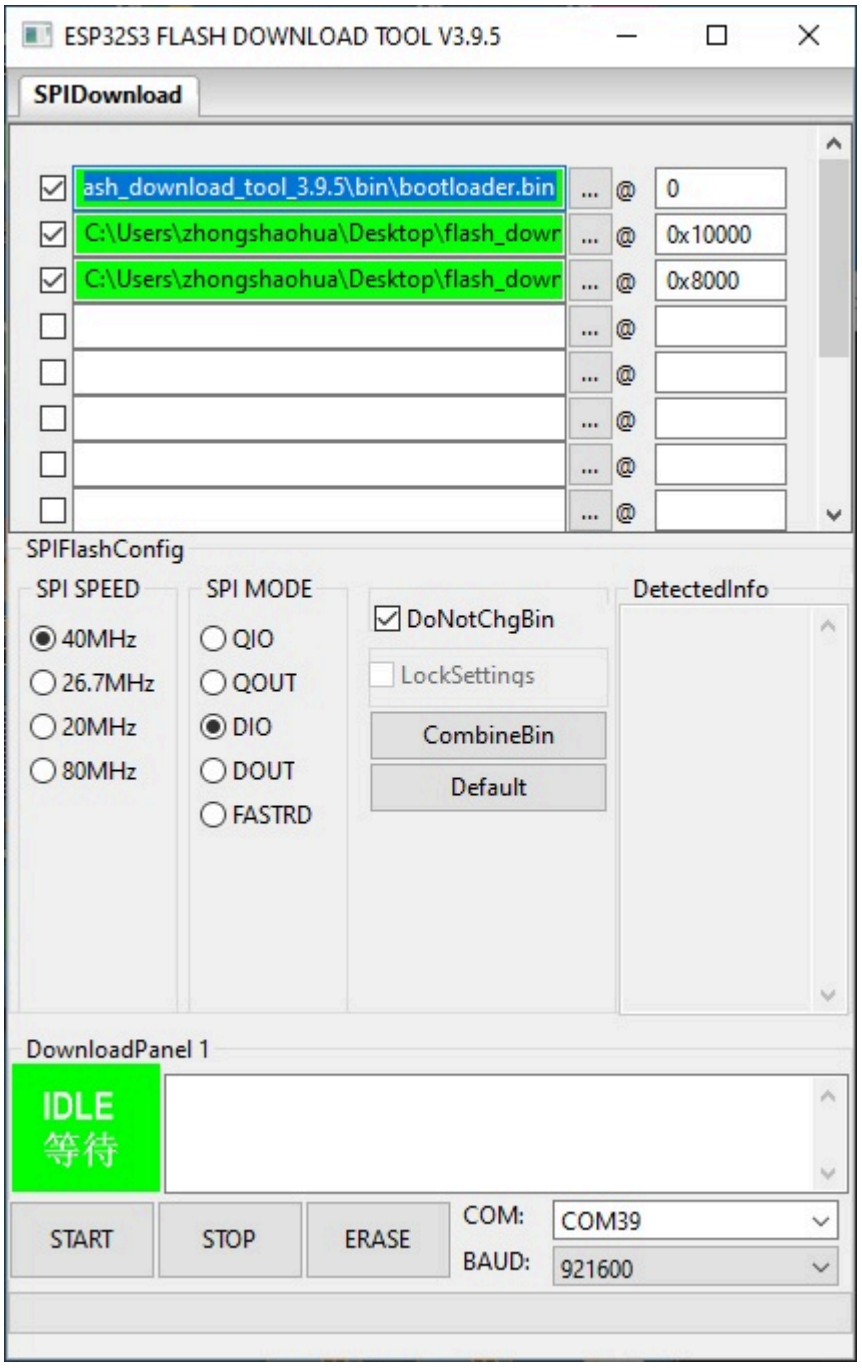
2. Press the BOOT key on the board, connect to the USB cable, and find the Device Manager or the corresponding COM port. Download or run the demo, and you can see the Hardware Description chapter for more details.

3. According to the steps below, download the online MPY firmware of the ESP32 series, and clean the Flash content of the development board before downloading, and the whole download process lasts about 1 minute.



(/wiki/File:ESP32-S2-Pico_022.jpg)

4. If the Tonny IDE needs to download the local firmware, you can operate it by following the steps below. Select Step 3 or Step 4, and Step 4 is recommended.

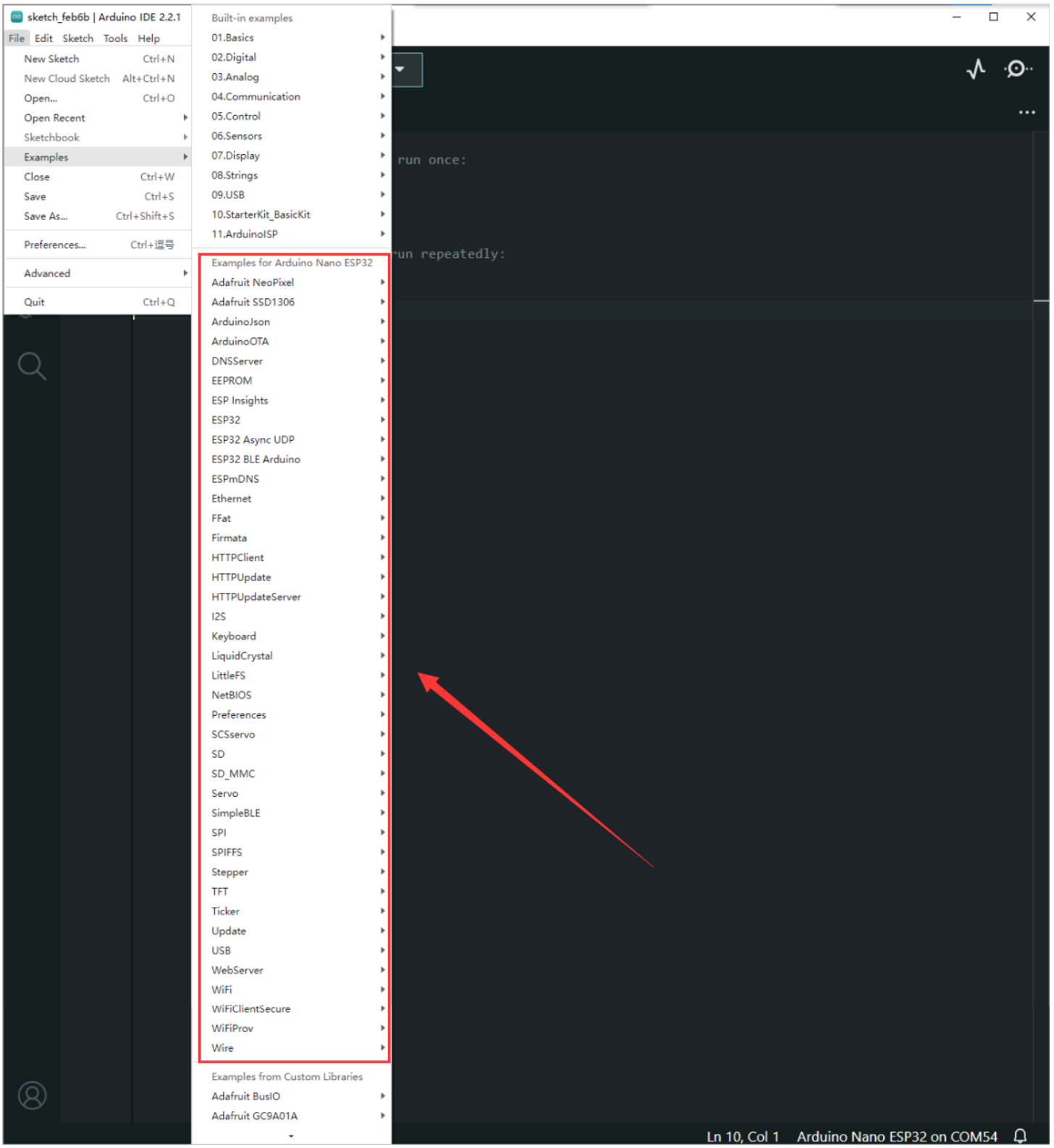


(/wiki/File:ESP32-C3-Zero_09.jpg)

5. Please refer to MicroPython Documentation (https://github.com/micropython/micropython/releases/tag/v1.18), releases note (https://github.com/micropython/micropython/releases/tag/v1.18) for programming.

Sample Demo

- For the Arduino example demo, please refer to arduino-esp32 (https://github.com/espressif/arduino-esp32) or File -> examples in Arduino IDE, these examples can be used directly without external libraries.



(/wiki/File:ESP32-S3-Nano_01.png)

- for mpy example, you can refer to MicroPython documentation (https://docs.micropython.org/en/latest/) and sample demo.

Resource

Document

- Schematic (https://files.waveshare.com/wiki/ESP32-S3-Nano/ESP32-S3-Nano-Schematic.pdf)
- MicroPython documentation (https://docs.micropython.org/en/latest/)

- ESP32 Arduino Core's documentation (https://docs.espressif.com/projects/arduino-esp32/en/latest/index.html)
- arduino-esp32 (https://github.com/espressif/arduino-esp32)

Demo

- Arduino sample demo (https://files.waveshare.com/wiki/ESP32-S3-Nano/ESP32-S3-Nano-Demo-Code.zip)

Software

- Sscom5.13.1.zip ()
- Thonny Python IDE (https://thonny.org/)
- Arduino IDE (https://www.arduino.cc/en/software)
- mpy firmware (https://files.waveshare.com/wiki/ESP32-S3-Nano/Esp32-s3-zero-mpy.zip)

Datasheet

- ESP32-S3 Datasheet (https://www.espressif.com/en/support/documents/technical-documents?keys=&field_type_tid%5B%5D=842)
- WS2812B (https://files.waveshare.com/wiki/ESP32-S3-Nano/XL-0807RGBC-WS2812B.pdf)

Support

Technical Support

If you need technical support or have any feedback/review, please click the **Submit Now** button to submit a ticket, Our support team will check and reply to you within 1 to 2 working days. Please be patient as we make every effort to help you to resolve the issue.
Working Time: 9 AM - 6 PM GMT+8 (Monday to Friday)

Submit Now ()